

## Part 14: Subroutine

1. **What is a subroutine in 8086?** It is a reusable block of code (like a function) that performs a specific task and can be called from anywhere in the main program.
2. **How do you define a subroutine?** Use a label followed by `PROC` (procedure) and end it with `RET` and `ENDP`.
3. **How do you call a subroutine?** Use the `CALL` instruction followed by the subroutine name (e.g., `CALL MyProc`).
4. **How do you return from a subroutine?** Use the `RET` (Return) instruction at the end of the subroutine.
5. **How do you pass parameters to a subroutine?** You can put values in registers (fastest) or push them onto the stack before calling.
6. **Which register is used for return address?** The return address is stored on the **Stack**, not in a specific general-purpose register.
7. **How is the stack used in subroutines?** The `CALL` instruction pushes the return address (IP) onto the stack, and `RET` pops it back to resume execution.
8. **How do local variables work in subroutines?** You can reserve space on the stack by subtracting from `SP` (Stack Pointer) or using `BP` (Base Pointer) to access that stack memory.
9. **Can a subroutine call another subroutine?** Yes, this is called nesting. The stack automatically manages the return addresses for each level.
10. **How do you implement recursion?** A subroutine calls itself. You must have a condition to stop calling (base case) so the stack doesn't overflow.
11. **How do flags behave in subroutines?** Subroutines generally affect flags just like any other code. If you need to preserve flags, `PUSHF` (Push Flags) and `POPF` (Pop Flags).
12. **How do you save registers before a subroutine call?** `PUSH` the registers you want to save onto the stack at the start of the subroutine.
13. **How do you restore registers after return?** `POP` the registers from the stack in the **reverse order** they were pushed just before the `RET` instruction.
14. **How do you use subroutines for code reuse?** Write the code once (e.g., a print function), and `CALL` it multiple times whenever needed instead of rewriting the instructions.
15. **How do you handle multiple return points?** It's best to have one exit label with the `RET` instruction and jump to it from other points in the subroutine.